

CORTEX — Components, Integrations, Data Flow, Build Map

1. Deployable Units (4 total)

- **ingestion-worker**
 - **fabric-api** (FastAPI: Fabric + Retrieval + Agents in one process)
 - **Next.js frontend**
 - **datastore containers**: Postgres, Neo4j, Qdrant, Redis
-

2. Components & Subcomponents

2.1 fabric-api (single FastAPI app)

- Query Classifier
- Context Assembly module
- Retrieval fan-out layer
 - Graph Traversal pathway
 - Vector pathway
 - Lexical pathway
- Structural Enrichment module
- Fusion + Re-rank module (local cross-encoder)
- LLM Generation module (citation resolution, self-check)
- Agent layer (LangGraph)
 - Supervisor
 - CORTEX Asset worker (QA/history, Recommendation, Compatibility/compare_specs, CGFR)
 - CORTEX Diagnose worker (Plan → Execute → Hypothesize → Self-critique → Stream)
 - CORTEX Comply worker (gap-check loop)
 - Shared tool: `context_graph_query`
- Auth/RBAC layer (JWT issuance, `require_role` guard)
- API routers: documents, query, entities, agents

2.2 ingestion-worker (Redis/RQ worker)

- Upload handler
- Docling parser (PDF native+scanned, XLSX/CSV via pandas/openpyxl)
- Entity + Spec Extraction
 - Regex tag-pattern pass
 - Fast-tier LLM JSON-schema extraction
 - P&ID VLM path (3b)
- Entity Resolution (2.5-stage: exact match → fuzzy match → embedding fallback)
- Knowledge Write stage
 - Postgres facts writer
 - Neo4j MERGE writer (nodes + edges)
 - Qdrant embed + upsert writer

2.3 Datastores

- **Postgres:** document_registry, facts, entity_registry, entity_aliases, users, audit_log, chunks (mirror table), FTS (tsvector/GIN)
- **Neo4j:** node labels (Equipment, FailureMode, MaintenanceAction, Procedure, Regulation, Document, Personnel, SparePart), edge types (relational + spec-type)
- **Qdrant:** single collection (chunk payload w/ bbox, doc_id, authority_class)
- **Redis:** ingestion job queue only (RQ)

2.4 Next.js Frontend

- AuthProvider / Layout
- GraphExplorer (Cytoscape.js) → sets `focused_tag` in GraphStore
- NodeSidePanel
 - CopilotChat (SSE/WS client)
 - MaintenanceTimeline
 - LinkedDocuments
 - AgentInvokeBar (AssetPanel, DiagnosePanel, ComplyPanel)
- ComplianceScoreView
- ReasoningOverlay (ReplayControls, ComparisonTable)
- GraphStore (centered node, expanded depth, highlighted paths, focused_tag)

3. Integrations Between Components

- ingestion-worker → Postgres (document_registry, facts writes)
 - ingestion-worker → Neo4j (MERGE node/edge writes)
 - ingestion-worker → Qdrant (embed + upsert)
 - ingestion-worker ← Redis (job dequeue)
 - fabric-api ↔ Postgres (facts, entity_registry, audit_log, users, FTS/lexical pathway)
 - fabric-api ↔ Neo4j (graph traversal pathway, `context_graph_query`, shortestPath, hub queries)
 - fabric-api ↔ Qdrant (vector pathway AND graph seed-expansion Strategy B — same collection, no second index)
 - fabric-api → Redis is NOT used for agent state (explicitly excluded)
 - Agents (Asset/Diagnose/Comply) → shared `context_graph_query` → Neo4j
 - Agents → `retrieve()` (fabric-api retrieval pipeline) → fan-out (Graph/Vector/Lexical)
 - fabric-api → LLM tiers: fast-tier (extraction, classification), frontier-tier (generation, agent reasoning, P&ID VLM) via direct SDK calls
 - frontend ↔ fabric-api: REST (`/api/documents`, `/api/entities/*`), SSE (`/api/query`), WS (`/ws/copilot/{session_id}`), agent endpoints (`/api/agents/*`)
 - frontend GraphStore.focused_tag → sent as payload field on `/api/query` and agent calls
 - Auth: frontend holds JWT in memory → Bearer token → fabric-api `decode_jwt` dependency → role-gated route guards
-

4. Data Flow

4.1 Ingestion Flow

```

Upload (multipart)
→ document_registry row (status=queued) + RQ job enqueued
→ Docling parse (native/scanned PDF, XLSX/CSV) → structured elements w/ bbox
→ Entity + Spec Extraction
    (regex tags) + (fast-tier LLM JSON extraction) + (P&ID VLM path for
drawings)
→ candidate (subject, predicate, object) triples
→ Entity Resolution (exact → fuzzy → embedding fallback) → resolved tags
→ Knowledge Write:
    Postgres: facts row (status=active)
    Neo4j: MERGE node + MERGE edge (fact_id, confidence, source_doc_id)
    Qdrant: chunk embedding + payload upsert
  
```

4.2 Query Flow (CORTEX Ask / Copilot)

```

Query + focused_tag + session_id
→ Query Classification (regex/keyword pre-filter, ambiguous → fast-tier LLM)
→ Context Assembly (explicit_tags from query, implicit_tags from chat
history + focused_tag, query_embedding)
→ Seed Expansion (Strategy A: direct tags; Strategy B: Qdrant top-10 →
facts.subject_tag; Strategy C: type-based, diagnostic only)
→ Fan-out (parallel):
    Graph Traversal pathway (shallow fixed-depth OR adaptive best-first/
decay/budget-bounded)
    Vector pathway (Qdrant top-20)
    Lexical pathway (Postgres FTS top-20)
→ Structural Enrichment (diagnostic/open only: multi-seed shortest paths +
hub detection) → synthetic passages
→ Fusion (query-type-weighted merge of graph/vector/lexical hits) + Re-rank
(local cross-encoder) → top 5-8 chunks
→ LLM Generation (frontier tier) → citation check → (regen once if
unresolved) → self-check faithfulness pass
→ Output: cited answer (SSE: token/citation/agent_trigger/done) → UI

```

4.3 Agent Invocation Flow

```

Query classified as diagnostic/compositional/compliance
    OR explicit endpoint hit (bypasses routing)
→ Supervisor (LangGraph) routes to exactly 1 of 3 workers
→ Worker calls shared context_graph_query (shallow/deep/auto) + retrieve()
→ Asset: history/QA, Recommendation, compare_specs, or CGFR chain
    (Constraint Extraction → Candidate Gathering → Compatibility Filtering →
Justified Selection)
→ Diagnose: Plan → Execute → Hypothesize → Self-critique → Stream (SSE,
replayable)
→ Comply: per-clause compliance-gap Cypher → aggregate score + red/amber/
green
→ Tier-1 (Respond, read-only) or Tier-2 (Propose, role-gated confirm/
dismiss) output
→ audit_log write

```

4.4 Feedback / Correction Flow

```

User correction via /api/entities/{tag}/feedback
→ new facts row (status=active)
→ old facts row marked status=superseded (superseded_by = new fact_id),
never deleted
→ Neo4j edge for old fact_id removed; new edge MERGED

```

5. What Has to Be Built

5.1 Data Layer (Track A)

- Postgres schema + migrations (001 init, 002 facts_fts, 003 facts_predicate_index)
- `document_registry`, `facts`, `entity_registry`, `entity_aliases`, `users`, `audit_log`, `chunks` mirror table
- FTS tsvector column + GIN index
- Neo4j node/edge model (all labels + relational + spec-type edge types)
- Neo4j MERGE write paths (node upsert, relational edge upsert, spec edge upsert, supersession edge removal)
- Neo4j rebuild/seed script (replay active Postgres facts)
- Qdrant single collection + chunk payload schema

5.2 Ingestion Pipeline (Track A)

- Upload handler + MIME/size validation + RQ enqueue
- Docling integration (PDF native+scanned, XLSX/CSV via pandas/openpyxl)
- Structure-aware parent-child chunking off Docling element boundaries
- Entity + Spec Extraction: regex tag pass, fast-tier LLM JSON-schema extraction (extended predicate set), P&ID VLM path
- 2.5-stage Entity Resolution (exact, fuzzy Levenshtein/Jaro-Winkler, embedding fallback)
- Knowledge write stage (Postgres + Neo4j + Qdrant)
- Error/fallback handling per pipeline stage (parse_failed, retry-once extraction, unresolved-mention logging, LLM timeout retry, P&ID VLM fallback to pre-loaded edges, CGFR zero-candidate explicit flag)
- Audit_log writes from ingestion side

5.3 Retrieval Pipeline (Track B)

- Query classifier (regex/keyword pre-filter + fast-tier LLM fallback)
- `assemble_context` (explicit/implicit tag resolution, chat-history mining, focused_tag plumbing)
- Seed expansion: Strategy A (direct), Strategy B (embedding-nearest via Qdrant), Strategy C (type-based, diagnostic only)
- Adaptive traversal (best-first, relevance-decayed, node-budget bounded, batched Cypher)
- Shallow fixed-depth traversal mode (permanent, not placeholder)
- Structural enrichment (multi-seed shortest paths, hub/degree heuristic)
- Passage synthesis (graph nodes → synthetic passages)
- Fusion weights per query type + local cross-encoder re-rank
- Citation resolution + regeneration loop + faithfulness self-check pass
- Performance tuning to budget table (context assembly, seed expansion, adaptive traversal, structural enrichment, passage synthesis latencies)

5.4 Agent System (Track C)

- LangGraph supervisor (routing logic: explicit endpoint bypass, implicit classification-based routing, fast-tier routing call)
- CORTEX Asset: shared node/tool registry across 4 capabilities (QA/history, Recommendation, compare_specs, CGFR)
- Tools: `get_entity_specs`, `find_by_type`, `compare_specs`
- CGFR chain: Constraint Extraction → Candidate Gathering → Compatibility Filtering → Justified Selection
- CGFR edge-case handling (no candidates, incomplete target specs, multiple passes, zero passes)
- CORTEX Diagnose: Plan → Execute → Hypothesize → Self-critique → Stream, max_iterations=5, insufficient-evidence fallback
- CORTEX Comply: per-clause gap Cypher, score aggregation, red/amber/green, out-of-scope-regulation handling, max_iterations=3
- Shared `context_graph_query` tool (shallow/deep/auto modes)
- Autonomy tier wiring (Tier-1 auto-log, Tier-2 propose + role-gated confirm/dismiss)
- max_iterations circuit breaker on all 3 workers
- Replay logging (sequence-numbered events)

5.5 API Surface (Track C/D)

- `POST /api/documents`, `GET /api/documents/{id}`
- `POST /api/query` (SSE: token/citation/agent_trigger/done)
- `GET /api/entities/{tag}`, `GET /api/entities/{tag}/graph`
- `POST /api/entities/{tag}/feedback`
- `POST /api/agents/diagnose`, `GET /api/agents/diagnose/{job_id}`, `GET /api/agents/diagnose/{job_id}/replay`
- `GET /api/agents/comply/gaps`
- `GET /api/agents/asset/risk/{tag}`, `GET /api/agents/asset/history/{tag}`
- `POST /api/agents/asset/recommend`
- `WS /ws/copilot/{session_id}`
- Uniform error envelope across all endpoints

5.6 Auth & RBAC (Track D)

- JWT issuance (bcrypt verify → sign {sub, role, exp})
- `require_role` dependency / route guards
- Role-based access matrix enforcement (Technician/Engineer/Compliance Officer/Admin)
- Personnel-name redaction below Engineer role
- Expired-token-mid-SSE-stream handling (401-equivalent SSE error, re-login without losing partial answer)

5.7 Frontend (Track D)

- `/login`, `/` (dashboard/graph explorer), `/graph`, `/entity/[tag]`, `/compliance`, `/agents/diagnose/[jobId]`, `/agents/asset/recommend` routes
- GraphExplorer (Cytoscape.js) with depth control, click-to-expand, focused_tag propagation
- CopilotChat SSE/WS client
- MaintenanceTimeline, LinkedDocuments panels
- AgentInvokeBar (Asset/Diagnose/Comply panels)
- ComplianceScoreView (single score + drill-down)
- ReasoningOverlay (agent event timeline) + ReplayControls
- ComparisonTable component (shared by compare_specs and CGFR final step)
- GraphStore (centered node, expanded depth, highlighted paths, focused_tag) — in-memory only, no localStorage/sessionStorage
- Streaming state buffering per session_id/job_id, flushed on animation frame
- Graph animations (re-centre, staged reveal, edge pulsing, dashed <0.7-confidence edges)
- Mobile-responsive pass (voice input stretch-only)

5.8 Deployment & Infra (Track D)

- docker-compose.yml (postgres, neo4j, qdrant, redis, fabric-api, ingestion-worker, frontend)
- Env var wiring (.env.example)
- Migration apply-on-startup sequencing (psql, ordered files)
- Seed script: users + canonical tag/alias list
- Bulk corpus ingestion CLI/loop
- Fallback mechanisms actually implemented and tested:
 - P&ID VLM slow/unreliable → pre-loaded curated drawing edges
 - Structural enrichment exceeds budget → drop, proceed on adaptive traversal alone
 - CGFR zero candidates → pre-seeded spare-parts catalog + explicit "no compatible candidate" response

5.9 Testing (all tracks)

- Unit tests: entity resolution triggers, fact/trust supersession transaction, context assembly, adaptive traversal decay/budget/propagation, structural enrichment bridge/hub logic, retrieval fusion weighting/renormalization, citation resolution paths, agent routing, CGFR constraint extraction + compatibility filtering, auth issuance/ expiry/route-guard
- Integration tests: seeded ~10-doc fixture corpus (incl. spare-parts catalog) against real Postgres/Neo4j/Qdrant; full ingestion pipeline count assertions; full Copilot lifecycle (shallow + full-traversal); all 3 agent workers incl. one full CGFR chain with known expected outcome
- E2E script (Playwright/Cypress): login → dashboard → equipment click → Diagnose invoke → Asset compositional question → compliance view → correction submit → graph-update assertion
- AI-specific validation: fast-tier extraction schema conformance on fixed sample set; prompt-diff review process for Copilot gen / Diagnose hypothesis / CGFR constraint+filtering prompts
- Acceptance-target test sets: retrieval precision set, citation spot-check set, CGFR test set, compliance gap set